

Внутриоблачные межсервисные вызовы

Участок: Маршрутизация запросов в схемы клиентов

Облако состоит из множества сервисов, которые постоянно взаимодействуют между собой. Одним из способов такого взаимодействия являются **внутриоблачные межсервисные вызовы** – обращения сервиса, входящего в инфраструктуру облака, к другому сервису из этой же инфраструктуры.

Понятие EndPoint

В межсервисном вызове всегда присутствует адресат вызова (**EndPoint**) – точка web-сервиса, которая принимает и обрабатывает вызов.

EndPoint является абстракцией, которая скрывает за собой физический адрес сервиса, протокол доставки, карту маршрутизации и т.п. Между собой EndPoint различаются только именами. В качестве имени EndPoint рекомендуется указывать системное имя сервиса.

Также можно использовать имя параметра из реестра [Настройки](#) или [ini-файла сервиса](#). Однако если даже существует сервис с таким же системным именем, то обращения к нему будут подавлены этим параметром. Поэтому данный способ именования EndPoint крайне нежелателен.



С помощью адресов EndPoint нельзя формировать ссылки. Почти всегда EndPoint содержит внутренний адрес сервиса, недоступный снаружи. Также этот адрес может измениться.

Вызовы методов из простых сервисов

Допустим, требуется вызвать метод "Тест.Эхо" с простого сервиса "Календарь", имеющего системное имя "calendar". Если "Календарь" – сервис без [аутентификации](#), то вызов будет выполнен с сессией родительского запроса, хотя самому сервису без аутентификации сессия не нужна. Вызов будет осуществлен с текущей сессией или без нее (при ее отсутствии).

- C++:

```
String result = bl::Object( L"Тест", L"calendar" ).Invoke<String>( L"Эхо", L"Ух" );
```

- Python:

```
1 # возможный вариант
2 result = BLObject('Тест', 'calendar').Invoke('Эхо', 'Фу-у...')
3 # оптимальный вариант
4 result = EndPoint('calendar').Тест.Эхо('Ок')
5 # "идеальный" вариант
6 calendar.Тест.Эхо('Ок')
```

Вызовы методов из multiversion-сервисов

Вызовы из [multiversion-сервисов](#) внешне аналогичны [вызовам из простых сервисов](#).

Если ваш multiversion-сервис реализуется на основе [version-контекста](#), то никаких специальных действий не требуется. В url вызова автоматически добавится параметр **x_version**, в котором хранится версия вызывающего, и на основе этой информации выбирается правильная реализация сервиса. Пример – сервис **specifications-ps**.

При реализации сервиса на основе [custom-контекста](#) возможны два варианта:

1. Если custom-контекст вычисляется от клиента/пользователя, к которому привязан, то в вызове обязательно должен фигурировать клиент.

2. Явное указание имени custom-контекста, например, контекст с именем "demo" в EndPoint(context = RouteContext('online', 'demo')).

Исключения:

- Получение адреса сервиса по его системному имени:

```
RemoteServiceCache.Instance().ShortServiceInfo( "kladr"
).Address(rpc::InteractionKind::SyncInvoke, a rpc::HTTP_PROTOCOL )
```

Данный способ не рекомендуется и может привести к ошибкам.

- При получении адреса сервиса по сценарию, политики указания клиента/пользователя можно использовать и для вызовов из простых сервисов.

Вызовы методов из multitenancy-сервисов

Для [multitenancy-сервисов](#) маршрутизация запросов выполняется под именем клиента, т.к. необходимо выбрать, на какую группу должен быть передан запрос. Если при обращении к сервису требуется сессия для получения информации о пользователе, то для выполнения вызова нужно в объекте EndPoint указать одну из политик аутентификации:

- **AuthByClientID(id)** – в **id** указывается только идентификатор клиента в облаке.
- **AuthByExtID(id)** – в **id** достаточно указать номер аккаунта из Биллинга, тратит время на вычисление клиента.
- **AuthByUserID(id)** – в **id** указывается идентификатор пользователя, тратит время на вычисление клиента.
- **AuthByProfileID(id, profile)** – в **id** указывается номер аккаунта в Биллинге, в **profile** – идентификатор персоны, тратит время на вычисление клиента и пользователя.
- **AuthByLogin(login)** – в **login** указывается логин пользователя, тратит время на вычисление клиента и пользователя.
- **AuthByClientUserID(client, user)** – в **client** указывается идентификатор клиента в облаке, **user** – идентификатор пользователя. Если в точке вызова известны оба идентификатора, то лучше пользоваться этим вариантом, он быстрее, чем **AuthByUserID**, т.к. не тратит время на вычисление клиента.

При этом передавать неявно сессию с родительского запроса **не рекомендуется** т.к. адрес в EndPoint может не совпадать с адресом сайта, для которого выдана сессия, и диспетчер в этом случае отбросит запрос с 401 ошибкой.



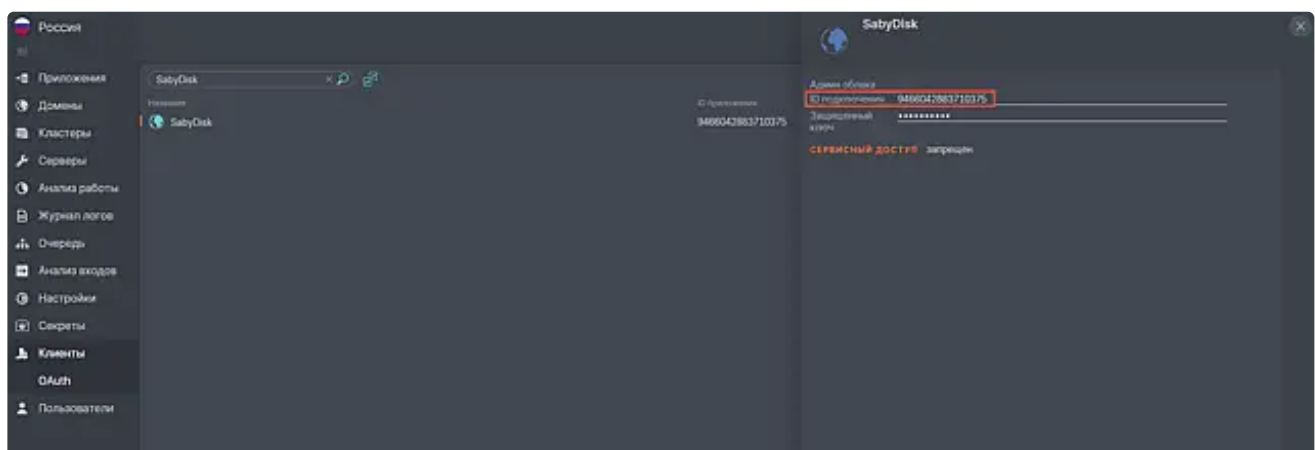
Данный способ вызова методов может использоваться и для [multiversion-сервисов](#).

Сейчас в нашем облаке к multitenancy относится сервис с системным именем "online". Также определенные сессии могут понадобиться для доступа к сервисам Диска, Биллинга и т. п.

Если удобнее выполнить вход через внешнее приложение, используйте следующие политики:

- **AuthByApplicationId(application_id)** - используется служебный пользователь приложения. В конкретном аккаунте (например, Тензор) нужно создать приложение и разрешить вход сервисному пользователю. Далее можно использовать политику в своем коде, она будет адресована созданному служебному пользователю приложения.
- **AuthByApplication(application_id, client_id)** - используются приложение и идентификатор клиента. В каждом аккаунте создается служебный пользователь, связанный с приложением.

Приложение можно добавить в разделе [Подключения к СБИС](#), либо в "Управлении облаком" ([реестр "Клиенты"](#)). Идентификатор приложения подставляется в политики в качестве **application_id**:



Карточка стороннего приложения в облаке

**Важно!**

Вызовы выполняются без проверки прав. При необходимости ее указывают в формате AuthByXXXXXX явно. Это позволяет ускорить выполнение вызова.

Примеры:

- C++:

```
1 #include <sbis-lib/sbis-cloud-  
  interaction/auth/user_id_auth_policy.hpp>  
2     bl::EndPoint ep( L"online" ), cloud_inter::AuthByUserID(1)  
  );  
3     String response = bl::Object( L"Тест", ep ).Invoke<  
  String >( L"Эхо", L"Ok" );
```

- Python:

```
1 # вариант вызова в старом стиле  
2 ep = EndPoint( service_name='online',  
  auth_data=AuthByUserID(1) )  
3 response = BLObject('Тест', ep).Invoke("Эхо", 'Ok' )  
4 ep = EndPoint( service_name='online',  
  auth_data=AuthByClientID(7) )  
5 response = BLObject('Тест', ep).Invoke("Эхо", 'Ok' )  
6 ep = EndPoint( service_name='online',  
  auth_data=AuthByClientUserID(7,1) )  
7 response = BLObject('Тест', ep).Invoke("Эхо", 'Ok' )  
8 ep = EndPoint( service_name='online', auth_data=AuthByExtID(7)  
  )  
9 response = BLObject('Тест', ep).Invoke("Эхо", 'Ok' )  
10 ep = EndPoint( service_name='online',  
  auth_data=AuthByLogin('my_login') )  
11 response = BLObject('Тест', ep).Invoke("Эхо", 'Ok' )  
12 # новый вариант вызова  
13 ep = EndPoint( service_name='online',  
  auth_data=AuthByUserID(1) )  
14 response = ep.Тест.Эхо('Ok')  
15 # вызов с проверкой прав  
16 ep = EndPoint( service_name='online',  
  auth_data=AuthByUserID(1,true) )  
17 response = ep.Тест.Эхо('Ok')
```